Final Submission Form

Student name: Boran Zhang (Bob)

Student ID: 2617285

Game title: Yoka's Adventure

Unity version: 2022.3.62f3c1

Build platform: Windows

GitHub repository link: https://github.com/starmatch9/3D-RPG-CuteStyleAdventureGame

Final commit hash: 1852cf7559fc4ae123f7abb5305f7ff72d9baf4c

Playable build link, if not uploaded directly: https://github.com/starmatch9/3D-RPG-CuteStyleAdventureGame/releases/tag/v0.0.1 and submitted directly as a ZIP file with this form.

How to run the game:

Download and extract the build folder.

Locate and double-click the “YuKa's Adventure.exe” file to launch the game.

Controls:

Action Keyboard / Mouse

Move W, A, S, D

Jump Space

Look Around / Rotate Camera Move Mouse

Pause / Resume Esc

Menu Navigation (Settings) Mouse Click

Main game objective:

Control the character to run and jump through the 3D world, collect enough water bottles, and reach the finish flag to complete the level.

Win / lose / completion condition:

Win Condition: Reach the finish flag. The number of water bottles collected

determines the star rating (1-3 stars) shown on the victory screen.

Lose / Fail Condition: If the character falls off the level, they will be respawned at the last activated checkpoint. There is no hard "game over" state.

Main systems/scripts I created:

Core Architecture: Generic Entity Framework & State Machine (EntityBase, Entity<T>, EntityState<T>, EntityStateManager<T>)

I designed a generic entity framework based on CRTP (Curiously Recurring Template Pattern). The Entity<T> base class allows each subclass—such as Player—to hold its own type at compile time, completely eliminating the overhead and risk of runtime type casting. On top of this, I implemented a type-safe state machine system where EntityState<T> and EntityStateManager<T> work together to support state switching by type, index, or instance, with onEnter, onExit, and onChange events automatically triggered during transitions. This state machine fully adheres to the Open/Closed Principle—adding a new state requires nothing more than inheriting from PlayerState and implementing the OnEnter/OnExit/OnStep methods, with zero modifications to existing code. The state list is presented as a dropdown menu in the Inspector through a custom [ClassTypeName] attribute (ClassTypeNameDrawer), allowing designers to select state classes directly without manually typing strings, completely eliminating the risk of typos.

Player Control System & Physics Feel (Player, PlayerStats, PlayerInputManager, PlayerStateManager)

The core of the player control system is the Player class, which encapsulates all behaviors including movement, jumping, gravity, friction, and ground snapping. The jump system implements Coyote Time (a brief window after leaving the ground where jump inputs are still accepted), Jump Buffer (caching jump inputs pressed just before landing), and variable jump height (holding for higher jumps, tapping for lower ones), delivering responsiveness comparable to Celeste. All feel parameters—including acceleration, turning drag, gravity values, jump height, and the number of double jumps—are stored in PlayerStats, a ScriptableObject data asset. This allows designers to freely adjust values in the Inspector without ever touching code, achieving a clean separation between data and logic. The input

system (PlayerInputManager) is built on Unity's Input System, simultaneously supporting keyboard, mouse, and gamepad, with cross deadzone and remapping smoothing that delivers a natural and refined feel for analog stick input. PlayerStateManager works with the [ClassTypeName] dropdown, allowing designers to configure the state list directly in the Inspector.

Camera System (PlayerCamera)

The camera system is built on Cinemachine, but I implemented three layers of fine-grained control on top of it. The first is manual orbiting—mouse and gamepad inputs are processed differently, with mouse input using Time.timeScale for instant responsiveness and gamepad input using Time.deltaTime for frame-rate-independent smooth rotation. The second layer is vertical dead zone following—dead zone sizes and follow speeds are independently configurable for ground and air states, with a tight dead zone (0.15) on the ground keeping the camera close to the character, and a larger dead zone (4.0) during jumps preventing the camera from shaking with the character's vertical motion. The third layer is velocity-driven rotation—when the character moves laterally, the camera automatically rotates slightly toward the direction of movement, creating a dynamic and fluid visual language.

Game Management & Interaction Systems (GameManager, Coin, GameSavePoint, GameFinish)

I unified game state management through the GameManager singleton, covering respawn point recording, collection counting, victory determination, and star rating. The collectible system (Coin) uses trigger collision—each water bottle collected plays a sound effect, increments the counter, and uses an exist flag to prevent duplicate triggering. The checkpoint system (GameSavePoint) allows designers to freely place flags in the scene; when the player touches one, the respawn position is updated and the flag activates visual feedback. The goal system (GameFinish) plays a victory sound effect upon player contact, triggers a flag scaling animation from 0x to 10x, and after a 2-second delay invokes the victory logic to light up 1 to 3 stars based on the collection count, with all thresholds fully configurable in the Inspector.

Audio System (EffectManager, BgmManager, GlobalData)

The audio system is divided into sound effects and background music. EffectManager provides global access through the singleton pattern, supporting

one-click playback of four categories of sound effects (UI, jump, pickup, and victory) with a single line of code: GlobalData.effectManager.PlayEffect(GlobalData.effectManager.eat). The background music manager BgmManager implements loop playback through coroutines with fade-in and fade-out effects, while also supporting configurable intervals and repeat counts for smooth musical transitions. GlobalData serves as a static global data container holding the EffectManager reference, eliminating the performance overhead of repeatedly looking up the singleton.

UI & Settings System (UI_Temp, SettingButtons, StopGame)

The UI system (UI_Temp) displays the collection counter in real time, with the victory panel dynamically lighting up 1 to 3 stars based on the number collected. The settings panel (SettingButtons) supports switching between three resolutions (1280×720, 1920×1080, 2560×1440) and toggling fullscreen/windowed modes, with all changes taking effect immediately upon clicking Save. The pause system (StopGame) controls game pause through Time.timeScale—when paused, physics, animations, coroutines, and particle systems are all frozen—while supporting a complete set of flow controls including resume, restart, return to main menu, and quit.

Custom Editor Tool (ClassTypeName, ClassTypeNameDrawer)

I developed a custom editor tool, ClassTypeNameDrawer, which scans the current application domain for all subclasses of a specified base class via reflection and presents them as a polished dropdown menu in the Inspector. Designers can directly select a state class from the dropdown without needing to remember class names or namespaces—this not only streamlines workflow efficiency but, more importantly, completely eliminates runtime errors caused by string typos. Newly added state classes appear in the dropdown automatically, achieving true zero-intrusion extension.

Animation Synchronization System (PlayerAnimator)

The animation synchronization system (PlayerAnimator) executes in LateUpdate, ensuring that the Animator receives the final state data of the current frame. I pass state indices and ground status to the Animator as precomputed hash values via Animator.StringToHash, avoiding per-frame string lookups and optimizing runtime performance while maintaining code readability. Parameter names are configurable

in the Inspector, improving adaptability across different Animator Controller setups.

External assets/templates/tutorials/AI used:

Assets: They can be used freely for non-commercial purposes.

Player: https://www.cgalpha.com/archives/68688.html

Skybox: https://gitcode.com/open-source-toolkit/528bd/?utm_source=tools_gitcode&index=top&type=card&&uuid_tt_dd=10_18490143710-1775296053067-663627&isLogin=1&from_id=142948307&from_link=8f3698f0580f73da2a95c6bbe7608969

Platform: https://kaylousberg.itch.io/block-bits/download/eyJleHBpcmVzIjoxNzgxMTA3MjYxLCJpZCI6MzM4OTc1NH0%3d.LMDxcbVBOSHK84av8NPuOzIf3Ww%3d

Animal: https://davidoreilly.itch.io/everything-library-animals

Bottle: https://indienova.com/resource/r/ddbxscyxscb-2020low-poly-survival-pack

Sound Effects: https://assetstore.unity.com/packages/audio/sound-fx/rpg-essentials-sound-effects-free-227708

BGM: https://www.haimian.com/create

AI: Used Haimian AI for background music generation(https://www.haimian.com/create).

Tutorials: Referenced Unity Learn tutorials on Input System and Cinemachine(https://docs.unity.cn/cn/2022.3/Manual/UnityManual.html).

Known issues:

Content & Completeness: Currently only one test level exists; lacks multi-level progression and difficulty curves. Overall playable content is limited, and the project is still far from a complete game product.

Gameplay Depth: Currently only features basic platforming and collection mechanics. Lacks enemies, time trials, hidden collectibles, or other replay-value elements. Players have little incentive to continue after completing the level.

Control Feel Tuning: Movement, jump, and camera parameters have not undergone sufficient user testing. Several values require further iteration for different player demographics.

Asset Replacement: Character model, skybox, and BGM are sourced from non-commercial or unclear licensing channels. Must be replaced with CC0 or properly commercial-licensed assets before any public release.

UI Polish: Current UI only implements basic functionality, lacking a unified art style and interactive feedback. The game also lacks a main menu and onboarding flow.